Differentiable Latent Indexing in LLMs: Toward Native Retrieval as a Cognitive Function

Author: Paulo H. Leocadio

Affiliation: Zinnia Labs

ORCID: https://orcid.org/0000-0002-4992-4541

Abstract

The problem addressed in this work is the persistent separation between parametric memory, encoded within model weights, and non-parametric memory retrieved through external databases or retrieval systems in modern Retrieval-Augmented Generation (RAG) architectures. Although RAG-based systems have improved factual grounding in large language models (LLMs), they introduce architectural fragmentation between the retrieval and generation components, which are often optimized independently and coupled only at inference time.

This separation introduces several limitations, including retrieval latency, semantic drift between retrieved evidence and model context, and restricted end-to-end gradient flow between evidence selection and downstream reasoning. Existing memory-augmented architectures partially address these issues but typically maintain retrieval as an external or weakly integrated process.

The DLI-LLM framework proposes an alternative architectural formulation in which differentiable retrieval mechanisms are integrated directly within Transformer layers through a latent key-value memory structure (Bahdanau, Cho, & Bengio, 2014). Retrieval queries are generated from hidden-state representations and routed through learned attention-gating mechanisms, enabling the retrieved vectors to participate directly within the transformer forward computation pathway.

The objective of this paper is not to claim empirical validation, but to formalize the architectural implications, theoretical properties, and comparative positioning of a retrieval-integrated transformer architecture within the broader landscape of retrieval-augmented and memory-augmented transformer architectures (Lewis, et al., 2020; Khandelwal, Levy, Jurafsky, Zettlemoyer, & Lewis, 2019; Jégou, Douze, & Schmid, 2011; Qu, 2021).

1. Introduction

Retrieval-Augmented Generation (RAG) architectures have improved factual grounding in large language models (LLMs) by combining parametric generation with external retrieval systems. However, these architectures frequently maintain an operational separation between retrieval and inference, producing architectural fragmentation, retrieval latency, and limited end-to-end differentiability between evidence selection and downstream generation.

This separation introduces several limitations, including retrieval latency, semantic drift between retrieved evidence and model context, and restricted end-to-end gradient flow between evidence selection and downstream reasoning. Existing memory-augmented architectures partially address these issues but typically maintain retrieval as an external or weakly integrated process.

The DLI-LLM framework proposes an alternative architecture in which differentiable retrieval mechanisms are integrated directly into Transformer layers via a latent key-value memory structure. Retrieval queries are generated from hidden-state representations and routed through learned attention-gating mechanisms, enabling the retrieved vectors to participate directly in the forward computation.

The objective of this paper is not to claim empirical validation, but to formalize the architectural implications, theoretical properties, and comparative positioning of this retrieval-integrated transformer architecture framework within the broader landscape of retrieval-augmented and memory-augmented transformer architectures.

1.1. Contributions

This work contributes:

- A conceptual architectural framework that integrates latent memory retrieval directly into transformer computation pathways.
- A mathematically formulated retrieval mechanism with differentiable routing and latent key-value memory integration.
- A theoretical blueprint for retrieval-integrated transformer architectures operating within the forward computation graph.
- A comparative analysis situating DLI-LLM relative to retrieval-augmented and memory-augmented transformer systems.
- A structured discussion of theoretical behaviors, architectural limitations, and future experimental directions associated with differentiable latent retrieval.

1.2. Overview of the Paper

Section 2 develops the methodological and architectural foundation of the DLI-LLM framework by decomposing the originating hypothesis into its principal architectural components. The section formalizes the retrieval mechanism through mathematical definitions, algorithmic specification, and implementation-oriented architectural modeling. It also situates the proposed framework in

relation to existing retrieval- and memory-augmented transformer architectures through comparative analysis.

Section 3 examines the predicted architectural behaviors and limitations of differentiable latent retrieval, including retrieval dynamics, memory scaling considerations, retrieval coherence, and potential failure modes, in the absence of empirical implementation.

Section 4 discusses broader implications for retrieval-integrated transformer architecture design, internal memory architectures, and future experimental investigations.

Section 5 presents the references.

Appendices A and B include the originating DLI-LLM hypothesis and the associated analytical synthesis material.

2. Architectural Framework and Methodology

This paper does not present empirical validation or benchmarked implementation results. Instead, it develops a conceptual architectural framework based on the Differentiable Latent Indexing inside Large Language Models (DLI-LLM) hypothesis, as reproduced in Appendix A. The methodology follows a structured process of architectural decomposition, mathematical formalization, comparative analysis, and implementation-oriented modeling to examine how differentiable retrieval mechanisms can be integrated directly into transformer computational pathways.

2.1. Architectural Foundations

The DLI-LLM framework is derived from four principal architectural assumptions:

A. The integration of a differentiable memory module within transformer layers.

B. The use of latent key-value memory structures to support scalable retrieval operations.

C. The existence of a shared latent vector space for token representations and retrieved document embeddings.

D. A learned routing and gating mechanism capable of dynamically controlling retrieval activation during inference.

Together, these components define a retrieval-integrated transformer architecture in which latent memory access becomes part of the forward computation pathway rather than an external retrieval stage.

Unlike conventional Retrieval-Augmented Generation (RAG) systems, in which retrieval and generation are often decoupled, DLI-LLM embeds retrieval directly into transformer-layer operations. Retrieval queries are generated from hidden-state representations and used to access latent memory structures through differentiable similarity functions. Retrieved vectors are subsequently fused back into the attention pathway through learned gating mechanisms.

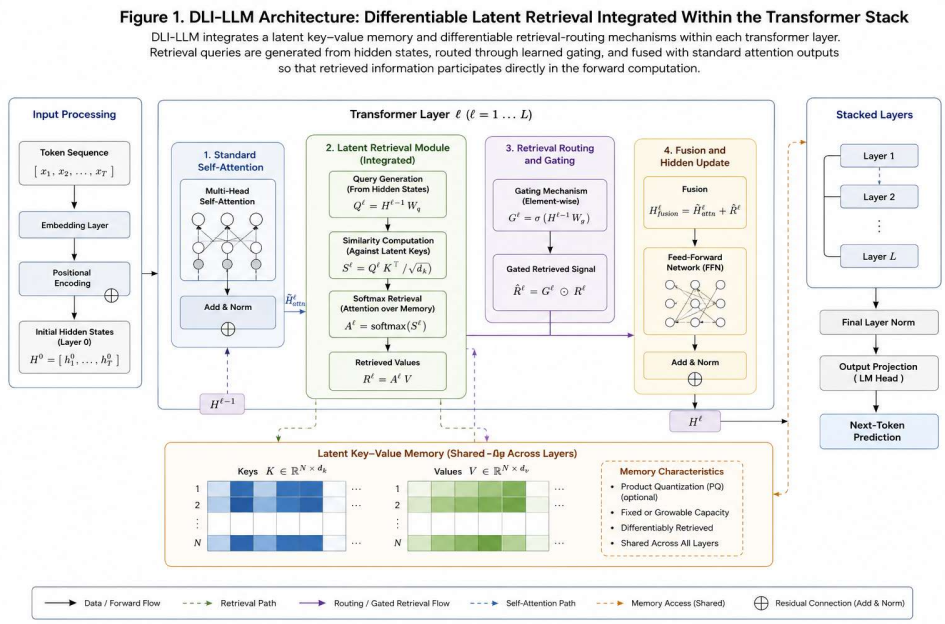


Figure 1. illustrates the conceptual retrieval pathway and latent memory integration process within the transformer stack.

2.2. Latent Memory Representation

The proposed framework assumes the existence of a latent key-value memory structure:

$$M = \{(k_i, v_i)\}_{i=1}^{N}, k_i, v_i \in \mathbb{R}^d$$

where k_i represents latent memory keys and v_i represents associated value vectors.

The architecture assumes that both token representations and retrieved document embeddings can coexist in a shared latent vector space, enabling retrieval operations to occur in the same representational domain as transformer hidden states.

To support scalability, the framework optionally incorporates product quantization strategies inspired by large-scale vector retrieval systems and differentiable memory architectures. This enables latent retrieval to remain computationally tractable even when memory capacity grows substantially.

2.3. Retrieval and Routing Mechanism

At each transformer layer, the hidden representation:

$$h_t \in \mathbb{R}^d$$

is projected into a latent retrieval query through a learned projection matrix:

$$q_t = W_q h_t$$

A similarity function computes retrieval scores between the query vector and latent memory keys:

$$s_i = \langle q_t, k_i \rangle$$

Differentiable retrieval weights are then computed through SoftMax normalization:

$$\alpha_i = \frac{\exp\,(s_i/\tau)}{\sum_j \exp\,(s_j/\tau)}$$

where τ represents a temperature parameter controlling retrieval sharpness.

The retrieved latent representation becomes:

$$r_t = \sum_{i=1}^{N} \alpha_i\, v_i$$

A learned routing gate modulates integration between retrieved memory and the original transformer hidden state:

$$g_t = \sigma(W_g[h_t; r_t])$$

The fused representation is subsequently computed as:

$$\tilde{h}_t = g_t \cdot r_t + (1 - g_t) \cdot h_t$$

This formulation enables retrieved latent memory vectors to participate directly in transformer-layer computation (Vaswani, et al., 2017) while preserving differentiable gradient propagation across both retrieval and attention pathways (Graves, Wayne, & Danihelka, Neural Turing Machines, 2014; Santoro, 2016; Graves, et al., 2016).

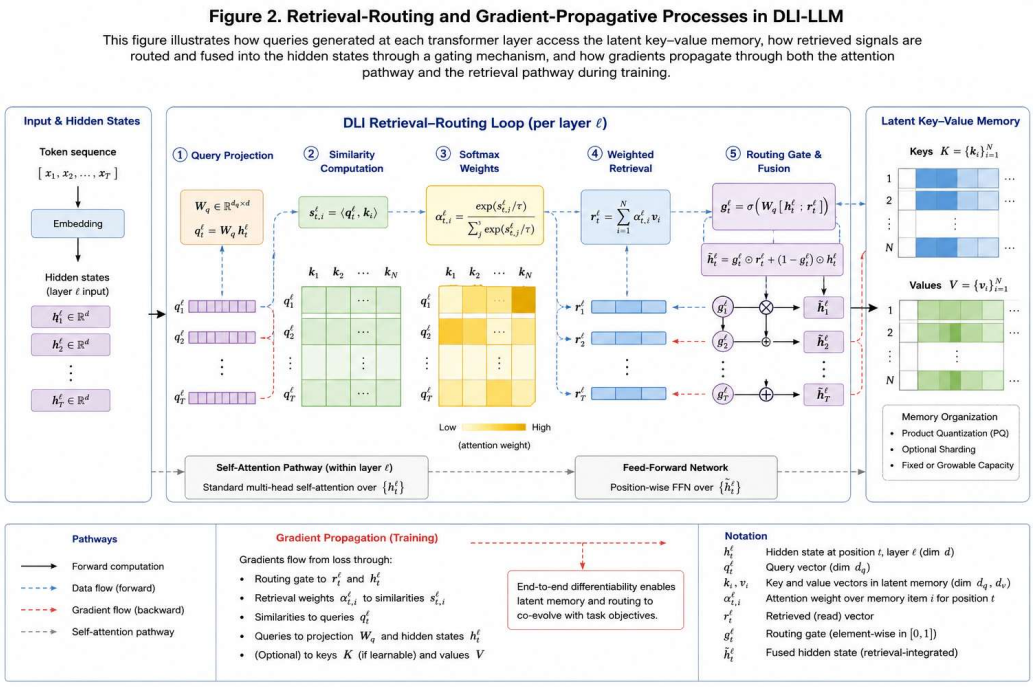


Figure 2. illustrates the retrieval-routing and gradient-propagation processes within the proposed framework.

2.4. Mathematical Interpretation

The proposed formulation extends standard transformer computation by introducing a differentiable retrieval pathway operating alongside self-attention mechanisms. Unlike external retrieval architectures, in which retrieval systems remain operationally separate from the language model itself, DLI-LLM integrates retrieval directly into the computational graph.

The framework draws conceptually from differentiable memory systems, attention-based addressing mechanisms, neural memory architectures, and retrieval-augmented transformer systemsn (Graves, et al., 2016). However, the distinguishing feature of DLI-LLM is the integration of latent retrieval directly within transformer-layer computation rather than through externally orchestrated retrieval pipelines.

The resulting architecture enables retrieval decisions, routing behavior, and memory integration to evolve jointly during optimization through shared gradient propagation.

2.5. Algorithmic Retrieval Process

Algorithm 1 summarizes a single retrieval operation within one transformer layer.

Input: Hidden state h_t, latent memory $M = \{(k_i, v_i)\}$, parameters W_q, W_g

Algorithm 1: Differentiable Latent Retrieval Step

Output: Fused representation $\tilde{h}_t$

A. $q_t \leftarrow W_q h_t$

B. Compute similarity scores $s_i = \langle q_t, k_i \rangle$

C. Compute retrieval weights $\alpha = \text{softmax}(s/\tau)$

D. Retrieve latent vector:

$$r_t = \sum_i \alpha_i \, v_i$$

E. Compute routing gate:

$$g_t = \sigma(W_g[h_t; r_t])$$

F.

$$\tilde{h}_t = g_t \cdot r_t + (1 - g_t) \cdot h_t$$

G. Return $\tilde{h}_t$

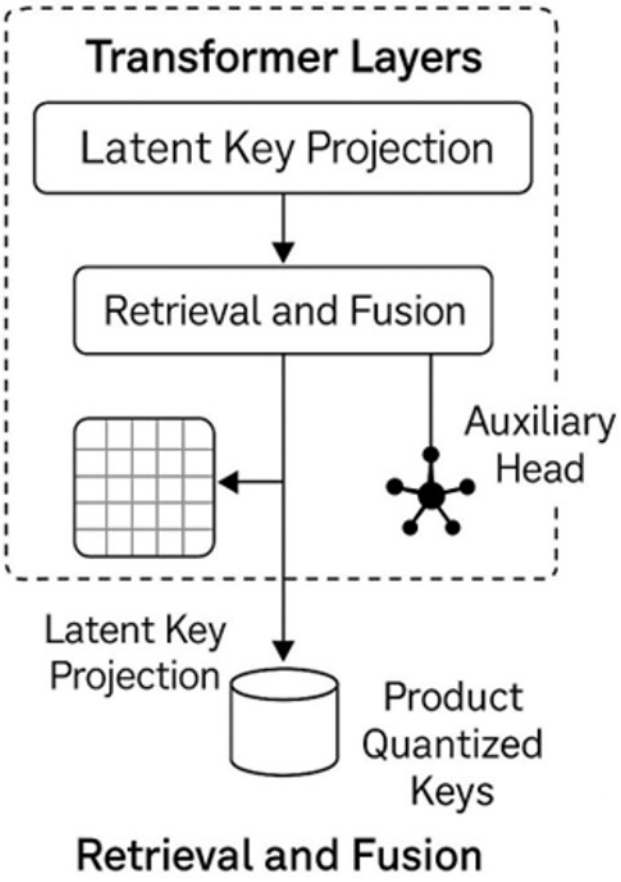


Figure 3. presents the conceptual computation graph associated with the retrieval-routing process.

2.6. Conceptual Implementation Framework

Although this work remains theoretical, an implementation-oriented DLI prototype would conceptually involve four principal stages:

Memory Construction

- Extraction of latent document embeddings
- Construction of latent key-value memory banks
- Optional application of product quantization strategies for scalable retrieval

Transformer Layer Integration

- Insertion of DLI retrieval blocks after self-attention layers
- Integration of learned routing and gating mechanisms
- Fusion of retrieved latent vectors within transformer-layer computation

Training Objective

A conceptual joint objective may combine language modeling and retrieval optimization:

$$\mathcal{L} = \mathcal{L}_{LM} + \lambda \mathcal{L}_{retrieval}$$

where λ controls the relative contribution of retrieval optimization.

Inference Behavior

Potential inference configurations may include:

- Frozen latent memory
- Sparse top-k retrieval
- Memory sharding
- External memory swapping mechanisms

2.7. Comparative Architecture Analysis

The DLI-LLM framework intersects conceptually with multiple memory-augmented and retrieval-integrated transformer architectures (Wu, 2022).

Architecture	*Relation to DLI-LLM*	*Limitation Relative to DLI-LLM*
Transformer-XL	Long-context recurrence	No latent key-value retrieval

RETRO	External retrieval augmentation	Retrieval remains externally orchestrated
Memorizing Transformers	Learnable memory tokens	No unified latent retrieval pathway
Differentiable Neural Computers	Key-value differentiable memory	Not integrated into transformer-layer computation
Recurrent Memory Transformers	Persistent memory states	Limited retrieval routing flexibility
KNN-LM	Post-hoc retrieval augmentation	Retrieval remains external and non-differentiable
Perceiver IO	Latent bottleneck architecture	No integrated latent retrieval routing
Hyena Hierarchy	Long-range implicit sequence modeling	No explicit memory retrieval mechanism

Table 1. Comparative analysis of memory-augmented and retrieval-integrated transformer architectures

2.8. Limitations and Theoretical Scope

This work does not present empirical implementation, benchmark evaluation, or experimental validation. The proposed framework should therefore be interpreted as a conceptual architectural model for formalizing differentiable latent retrieval within transformer-based systems.

Several limitations remain unresolved, including:

- computational scaling of latent memory structures,
- memory saturation and retrieval interference,
- retrieval interpretability,

- training stability,
- and the computational overhead associated with differentiable memory routing.

The framework also assumes that it is feasible to jointly optimize retrieval and language modeling objectives within a unified transformer computation graph, an assumption that requires future empirical validation.

Accordingly, the present work should be understood as a theoretical systems architecture proposal designed to motivate future implementation-oriented and experimental investigation into retrieval-integrated transformer architecture models.

3. Predicted Architectural Behaviors

Because this work does not present empirical implementation or benchmark evaluation, the behaviors discussed in this section should be interpreted as theoretical architectural predictions derived from the proposed DLI-LLM framework and related findings in prior memory-augmented language model research.

3.1. Predicted Retrieval and Memory Behaviors

3.1.1. Retrieval Efficiency

By integrating retrieval directly within the transformer's forward pass, DLI-LLM may reduce the external retrieval bottlenecks commonly associated with Retrieval-Augmented Generation (RAG) architectures (Wu, 2022). This prediction is consistent with prior observations that reducing external retrieval overhead can improve retrieval efficiency and inference coordination in memory-augmented systems.

3.1.2. Grounding Stability and Hallucination Reduction

The integration of latent memory retrieval pathways may improve contextual grounding during generation by enabling retrieved latent representations to participate directly in transformer-layer computation. Similar retrieval-assisted grounding effects have been observed in retrieval-augmented generators and nearest-neighbor language models.

3.1.3. Memory-Assisted Generalization

Because latent memory retrieval is integrated differentiably within transformer layers, the framework may exhibit memory-assisted generalization similar to that observed in hybrid retrieval and memory-augmented architectures.

3.1.4. Few-Shot Adaptation Dynamics

The proposed routing and gating mechanisms may improve adaptation under limited supervision by selectively integrating retrieved latent representations according to contextual relevance. Similar dynamics have been reported in memory-augmented neural systems employing differentiable memory access.

3.1.5. Long-Context Retrieval Coherence

Cross-layer retrieval propagation may improve retrieval consistency across extended token sequences by allowing latent memory interactions to evolve throughout transformer-layer computation. This behavior conceptually aligns with trends observed in long-context and persistent memory transformer architectures (Bulatov, Burtsev, & Kuratov, 2022).

3.2. Predicted Failure Modes and Architectural Risks

3.2.1. Memory Saturation

As with other memory-augmented architectures, finite latent memory capacity may lead to retrieval saturation under large-scale document loads or prolonged retrieval accumulation.

3.2.2. Retrieval Redundancy

High-dimensional latent retrieval may produce semantically redundant value-vector retrieval patterns, potentially reducing retrieval precision without appropriate compression or quantization strategies.

3.2.3. Contextual Retrieval Drift

Cross-layer retrieval propagation may introduce retrieval-context misalignment when latent query representations diverge from earlier contextual states. Similar forms of retrieval instability have been observed in persistent memory architectures.

3.2.4. Retrieval Poisoning and Memory Contamination

Because latent retrieval pathways participate directly in transformer computation, corrupted or adversarial memory representations may propagate erroneous retrieval signals throughout downstream generation pathways. This introduces potential risks associated with latent memory contamination and retrieval integrity.

3.3. Comparative Architectural Behavior Analysis

Figure 4 presents a conceptual comparison between DLI-LLM and conventional retrieval-augmented architectures across multiple architectural dimensions, including retrieval coordination, grounding stability, memory coherence, and the distribution of retrieval failures.

The figure does not present empirical benchmark data and should instead be interpreted as a theoretical comparative model derived from architectural analysis and prior observations in memory-augmented transformer research.

4. Discussion

The DLI-LLM framework proposes a retrieval-integrated transformer architecture in which latent memory access is incorporated into the forward-computation pathway rather than into an externally orchestrated retrieval stage. This architectural formulation has implications for how memory coordination, retrieval dynamics, and contextual grounding may be modeled within transformer-based systems.

4.1. Retrieval Integration and Memory Coordination

By embedding differentiable latent retrieval directly within transformer layers, the proposed framework reduces the operational separation between retrieval and inference commonly observed in Retrieval-Augmented Generation (RAG) architectures. Similar principles have appeared in memory-augmented neural systems and differentiable memory architectures. However, DLI-LLM extends these concepts by integrating shared latent retrieval pathways that operate directly within transformer-layer computation.

The proposed framework suggests that retrieved latent representations may participate directly in contextual computation rather than functioning as externally injected retrieval artifacts.

4.2. Learning Dynamics and Adaptation

The framework also suggests potential implications for continual learning and adaptive memory integration (Jaeger, 2007). Because latent memory structures can be updated independently of full

model retraining, retrieval-integrated transformer architectures could support incremental memory adaptation amid evolving information conditions.

At the same time, this flexibility introduces architectural challenges associated with memory scaling, retrieval interference, latent redundancy, and retrieval saturation. Efficient pruning, quantization, and routing strategies will likely be necessary to maintain stable retrieval behavior at scale.

4.3. Bias, Safety, and Retrieval Integrity

Integrating retrieval pathways directly within transformer computation also introduces safety and interpretability considerations. Latent memory structures may preserve biased, corrupted, or privacy-sensitive representations that become difficult to inspect once integrated into differentiable retrieval pathways.

These concerns partially resemble challenges observed in retrieval-augmented systems. Still, they may become more difficult to isolate when retrieval coordination occurs internally through latent routing mechanisms rather than through externally inspectable retrieval pipelines.

4.4. Routing and Architectural Control

A central component of the proposed framework is the retrieval-routing gate, which regulates how retrieved latent vectors influence transformer hidden states. This routing process determines the degree to which retrieved memory participates in downstream computation and contextual inference.

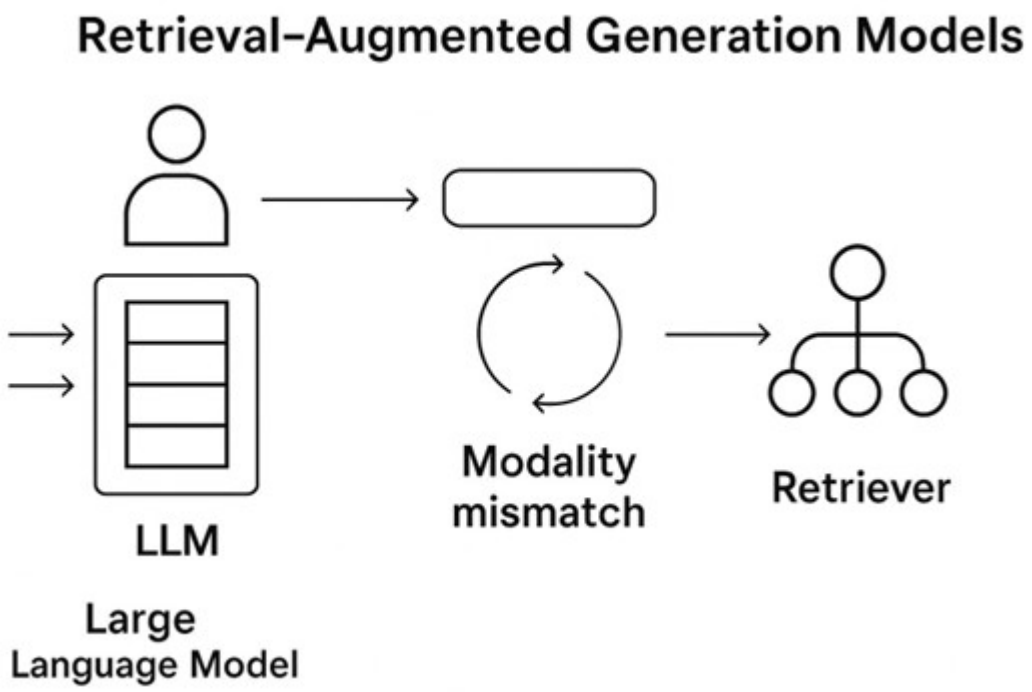


Figure 4. Conceptual illustration of retrieval fragmentation and modality separation in conventional retrieval-augmented architectures.

4.5. Broader Architectural Implications

More broadly, DLI-LLM contributes to ongoing research exploring how retrieval, memory coordination, and contextual inference may become more tightly integrated within transformer-based architectures. Rather than treating retrieval exclusively as an external augmentation layer, the framework proposes a conceptual direction in which latent retrieval mechanisms participate directly in model computation.

Although the present work remains theoretical and requires empirical validation, it provides a formal architectural basis for future investigation into retrieval-integrated transformer architectures and differentiable latent memory coordination.

5. Conclusion

This paper formalized the DLI-LLM hypothesis as a conceptual architectural framework for integrating differentiable latent retrieval directly within transformer-layer computation. Unlike conventional retrieval-augmented architectures that maintain an operational separation between retrieval and inference, the proposed framework introduces a unified latent retrieval pathway operating within the transformer's forward pass.

The work presented a mathematical formulation of differentiable latent retrieval, an implementation-oriented architectural model, and a comparative analysis relative to existing memory-augmented and retrieval-integrated transformer systems. In addition, the paper examined predicted architectural behaviors, retrieval dynamics, and potential limitations associated with latent memory integration.

Although the framework remains theoretical and requires empirical implementation and benchmark validation, the proposed architecture provides a structured basis for future investigation into retrieval-integrated transformer architectures, differentiable memory coordination, and latent retrieval mechanisms operating directly within model computation pathways.

6. References

Bahdanau, D., Cho, K., & Bengio, Y. (2014, September). Neural Machine Translation by Jointly Learning to Align and Translate. *International Conference on Learning Representations.* arXiv. doi:10.48550/ARXIV.1409.0473

Bulatov, A., Burtsev, M., & Kuratov, Y. (2022). Recurrent Memory Transformer. *Advances in Neural Information Processing Systems 35* (pp. 11079-11091). Neural Information Processing Systems Foundation, Inc. (NeurIPS). doi:10.52202/068431-0805

Graves, A., Wayne, G., & Danihelka, I. (2014). Neural Turing Machines. doi:10.48550/ARXIV.1410.5401

Graves, A., Wayne, G., Reynolds, M., Harley, T., Danihelka, I., Grabska-Barwińska, A., . . . Hassabis, D. (2016, October). Hybrid computing using a neural network with dynamic external memory. *Nature, 538*, 471–476. doi:10.1038/nature20101

Jaeger, H. (2007). Echo state network. *Scholarpedia, 2*, 2330. doi:10.4249/scholarpedia.2330

Jégou, H., Douze, M., & Schmid, C. (2011, January). Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence, 33*, 117–128. doi:10.1109/TPAMI.2010.57

Johnson, J., Douze, M., & Jegou, H. (2021, July). Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data, 7*, 535–547. doi:10.1109/TBDATA.2019.2921572

Khandelwal, U., Levy, O., Jurafsky, D., Zettlemoyer, L., & Lewis, M. (2019, November). Generalization through Memorization: Nearest Neighbor Language Models. doi:10.48550/ARXIV.1911.00172

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., . . . Kiela, D. (2020, May). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. doi:10.48550/ARXIV.2005.11401

Qu, Y. D. (2021). An optimized training approach to dense passage retrieval for open-domain question answering. *arXiv pre-print*, pp. 5835–5847. Retrieved from http://arxiv.org/abs/2010.08191

Santoro, A. B. (2016). One-shot Learning with Memory-Augmented Neural Networks . *arXiv pre-print*. Retrieved from http://arxiv.org/abs/1605.06065

Sun, R. (2004, September). Desiderata for cognitive architectures. *Philosophical Psychology, 17*, 341–373. doi:10.1080/0951508042000286721

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., . . . Polosukhin, I. (2017). Attention Is All You Need. In *Advances in Neural Information Processing Systems (NIPS* (p. 30). arXiv. doi:10.48550/ARXIV.1706.03762

Wu, Y. R. (2022). Memorizing Transformers. *arXiv pre-print*. Retrieved from http://arxiv.org/abs/2203.08913

APPENDIX A

An AI generated a novel hypothesis for you:

Differentiable Latent Indexing inside Large Language Models (DLI-LLM)

Problem

Traditional retrieval engines sit outside the LLM and operate on separate embeddings, leading to bottlenecks and modality mismatch.

Motivation

Embed an on-the-fly, differentiable key-value index directly inside the transformer layers, allowing training to jointly learn what keys to store and how to retrieve.

Proposed Method

Differentiable Latent Index LLM (DLI-LLM) augments each transformer block with a learnable associative memory. During pre-training, every output token is paired with a latent key (projected hidden state) and stored in a product-quantized key table. Retrieval queries are generated by an auxiliary head that predicts when external evidence is needed. The memory returns the top-k value vectors, which are fused back into attention via cross-attention.

To bridge long contexts, we feed document chunk embeddings generated at ingestion time into the same latent space. Because both token memories and document memories live in identical vector spaces, the model can seamlessly interleave parametric and non-parametric knowledge. Gradient-based updates allow the LLM to tweak both keys and the attention gating function.

At inference, the index is frozen and can be sharded across GPUs for billion-scale corpora. A sparsity-promoting loss limits the number of retrieval calls. Fine-tuning on QA tasks teaches the model to prefer latent index hops over hallucination when evidence is needed.

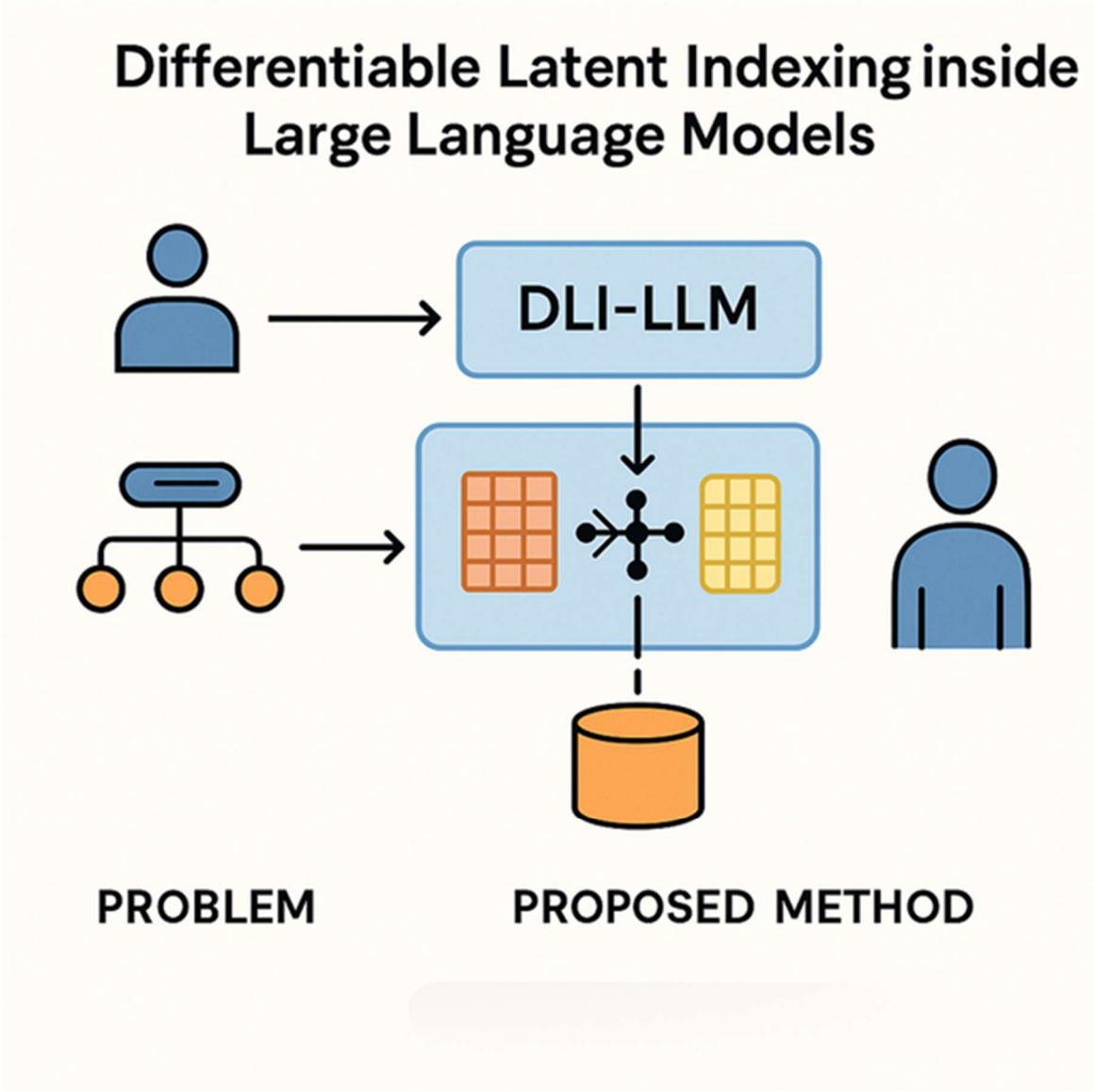
This architecture essentially blurs the line between RAG and long-context processing: the model internally decides whether to recall its own training memories or look up freshly indexed documents, all inside the forward pass.

Experiment Plan

Implement on top of 7-B and 13-B open LLMs. Compare to external RAG baselines on TriviaQA, HotpotQA, and BioASQ. Measure latency, GPU memory, and retrieval recall. Conduct ablations removing latent index or freezing it. Evaluate hallucination rate on fact-checking tasks.

Think of giving an AI a built-in Rolodex where it can jot down important facts and instantly flip to them while talking, instead of calling another service to look things up. DLI-LLM stitches the Rolodex inside the AI's 'brain,' making look-ups faster and more natural.

Non-Expert Description



How we generate hypotheses for you:

1. Our system digests 100 million peer-reviewed papers to build the knowledge base.
2. A large language model then uses this knowledge base to generate 92,000 draft ideas, using about 30 million tokens of generation.
3. The drafts face 23,000 rapid AI match-ups to choose the top 50 ideas—a 0.05% acceptance rate.

Rate the novelty of this hypothesis

Lock

PRIVATE TO YOU

1 **Not novel at all**

No meaningful differences from existing ideas; identical to many prior works.

2 **Very low novelty**

Minimal and unsubstantial differences; closely resembles existing ideas.

3 **Low novelty**

Minor differences present, but the hypothesis remains highly similar to existing ideas.

4 **Slight novelty**

Some differences from existing ideas, but overall still closely aligned.

5 **Moderate novelty**

Clear differences from existing ideas, but likely not enough to form a distinct paper.

6 **Reasonable novelty**

Notable differences from existing ideas; potentially sufficient to support a new paper.

7 **Good novelty**

Distinct differences from most existing ideas, offering potential for new contributions.

8 **Strong novelty**

Major differences from all known ideas; clearly stands apart from existing work.

9 **Very strong novelty**

High originality with minimal resemblance to existing ideas; notably innovative.

10 **Exceptional novelty**

> Fundamentally different from all existing ideas in a uniquely clever and compelling way.

Add your comments (optional)

This idea integrates memory retrieval into the transformer architecture itself, forming a hybrid between long-context modeling and retrieval-augmented generation. It proposes a fundamentally different architecture for knowledge grounding—differentiable, latent, and self-managed—within the LLM's forward pass.

Submit